

Session 08

Project Task RBAC API

আজকের implementation-এর বিস্তারিত ব্যাখ্যা

বিষয়	তথ্য
তারিখ	22 September 2026
প্রজেক্ট	FastAPI + PostgreSQL Project Task Management API
মূল বিষয়	Authentication, JWT, RBAC, Projects, Task Assignment, Limits
উদ্দেশ্য	নিজে বোঝা এবং অন্যকে সহজে বুঝিয়ে বলা

1. আজকের কাজের সংক্ষিপ্ত ফলাফল

আজ একটি সাধারণ task/user CRUD API-কে role-based project task management API-তে রূপান্তর করা হয়েছে। এখানে user login করে JWT token পায়, protected route token যাচাই করে, admin আলাদা ক্ষমতা পায়, project owner নিজের project পরিচালনা করে এবং normal user শুধু নিজের assigned task দেখে।

Requirement	কীভাবে পূরণ করা হয়েছে
Authentication dependency	get_current_user dependency সব project, task ও admin route-এ বসানো হয়েছে।
Admin route আলাদা	routers/admin.py-তে /admin prefix ব্যবহার করা হয়েছে।
RBAC	User.role দিয়ে admin check; project.owner_id দিয়ে owner check; ProjectMember.can_edit দিয়ে edit check।
Deactivate / delete	Admin user deactivate, activate এবং delete endpoint পেয়েছে।
Owner সব task দেখবে	Project owner project-এর সব task list করতে পারে।
User assigned task দেখবে	Non-owner list query assigned_to_id == current_user.id দিয়ে filter হয়।
Owner edit permission দেবে	ProjectMember table ও permission update endpoint যুক্ত হয়েছে।
Admin limits	SystemLimit table এবং /admin/limits endpoint যোগ হয়েছে।

2. পুরো request flow এক নজরে

```
Register -> hashed password -> users table
Login -> password verify -> JWT access_token
Bearer token -> get_current_user -> active user check
Admin route -> require_role('admin')
Project route -> owner/member permission check
Task route -> project permission + limit check -> database
```

এখানে FastAPI Dependency Injection সবচেয়ে গুরুত্বপূর্ণ। Route function-এর parameter-এ Depends(...) লিখলে FastAPI route চালানোর আগে সেই security/database function চালায়। ফলে একই security logic বারবার লিখতে হয় না।

3. Packages এবং কেন ব্যবহার করা হয়েছে

Package	কাজ	কোডে কোথায়
fastapi	API route, request validation, dependency injection, HTTP error/response তৈরি করে।	main.py, routers/*.py, core/deps.py
uvicorn[standard]	FastAPI application চালানোর ASGI server। --reload দিলে code বদলালে server restart হয়।	Command: python -m uvicorn main:app --reload
sqlalchemy	Python class-কে database table বানায় (ORM), query ও async session দেয়।	models.py, db.py, routers/*.py
asyncpg	PostgreSQL-এর asynchronous driver; SQLAlchemy-এর async engine এটি ব্যবহার করে।	DATABASE_URL-এর postgresql+asyncpg অংশ
python-dotenv	.env থেকে DB credential এবং SECRET_KEY load	db.py, core/security.py

Package	কাজ	কোডে কোথায়
	করে।	
pwdlib[argon2]	Password plain text-এ না রেখে Argon2 hash তৈরি এবং verify করে।	core/security.py, routers/auth.py
python-jose[cryptography]	JWT encode/decode ও signature validation করে।	core/security.py
pydantic (FastAPI dependency)	Request body validate এবং response shape নিয়ন্ত্রণ করে।	schemas.py

4. Environment configuration: .env

.env file-এ database connection তথ্য এবং JWT secret রাখা হয়। এটি source code-এ hard-code করা নিরাপদ নয় এবং .gitignore-এ রাখা উচিত।

```
DB_NAME=offline3db
DB_USER=postgres
DB_PASSWORD=...
DB_HOST=localhost
DB_PORT=5432
SECRET_KEY=replace-with-a-long-random-secret
```

- SECRET_KEY অন্য কারও সঙ্গে share করা যাবে না। এটি বদলালে পুরোনো JWT token invalid হয়ে যাবে।
- Production-এ strong random secret ব্যবহার করতে হবে; example text রেখে দেওয়া যাবে না।

5. core/security.py: password ও JWT security

এই file-এর কাজ দুইটি: password hash/verify করা এবং signed JWT token তৈরি/যাচাই করা।

লাইন	কী করে	কেন দরকার
3	import os	Environment variable পড়তে os.getenv ব্যবহার করা হয়।
4	datetime, timedelta, timezone import	Token expiry সময় UTC-তে নির্ভুলভাবে হিসাব করা হয়।
6	load_dotenv import	.env file থেকে SECRET_KEY load করে।
7	JWTError, jwt import	Token encode/decode এবং invalid token error ধরার জন্য।
8	PasswordHash import	pwdlib-এর password hashing object।
10	load_dotenv()	File import হওয়ার সময় .env values process environment-এ আনে।
12-14	SECRET_KEY পড়া ও validation	Key না থাকলে app silent insecure অবস্থায় না চলে, পরিষ্কার RuntimeError দেয়।
16-17	HS256 ও 60 মিনিট expiry	JWT signature algorithm এবং token validity period নির্ধারণ করে।
20	PasswordHash.recommended()	বর্তমান recommended secure password hasher (Argon2) তৈরি করে।

লাইন	কী করে	কেন দরকার
23-25	hash_password	Registration-এর plain password hash করে database-এ রাখে।
28-30	verify_password	Login-এর password stored hash-এর সঙ্গে মিলিয়ে true/false দেয়।
35-39	create_access_token	data copy করে exp claim যোগ করে signed JWT string দেয়।
41-45	decode_access_token	Signature ও expiry verify করে payload দেয়; invalid হলে None দেয়।

JWT payload-এর sub field-এ user id string হিসেবে রাখা হয়েছে। exp field JWT-এর standard expiry claim।

6. core/deps.py: reusable authentication ও role dependency

লাইন	ব্যাখ্যা
2-7	FastAPI dependency tools, async database type, User model ও token decoder import করা হয়েছে।
9	OAuth2PasswordBearer বলে দেয় যে Authorization header থেকে Bearer token পড়তে হবে এবং Swagger login endpoint কোথায়।
11	get_current_user হল প্রধান authentication dependency। Depends(oauth2_scheme) token নেয় এবং Depends(get_db) database session নেয়।
12-14	JWT decode করে; invalid বা expired হলে 401 Unauthorized দেয়।
15-21	payload থেকে sub নেয় এবং int-এ convert করে। ভুল format হলে 401 দেয়, server error নয়।
22-24	sub-এর user id দিয়ে database থেকে current user নেয়; user মুছে গেলে 401 দেয়।
25-29	is_active false হলে 403 এবং task-এর required message দেয়: Your account is temporarily banned।
30	সব check pass করলে User object route function-এ ফেরত দেয়।
33-41	require_role('admin') factory function। এটি checker dependency বানায় এবং current_user.role allowed role-এর মধ্যে আছে কি না যাচাই করে।

401 মানে identity/token সঠিক নয়; 403 মানে user চেনা গেছে কিন্তু তার permission নেই। এই পার্থক্য API design-এ গুরুত্বপূর্ণ।

7. Database connection ও startup migration: db.py

লাইন	ব্যাখ্যা
1-2	text দিয়ে raw SQL চালানো হয়; create_async_engine/async_sessionmaker দিয়ে async PostgreSQL connection তৈরি হয়।
4-14	.env load করে DB_NAME, DB_USER, DB_PASSWORD, DB_HOST, DB_PORT নেওয়া হয়।
17	PostgreSQL async connection string তৈরি হয়: postgresql+asyncpg://user:password@host:port/db।
20	engine হল connection factory। echo=True এখন development debugging-এর জন্য SQL log দেখায়। Production-এ সাধারণত False করা উচিত।
23	AsyncSessionLocal প্রতি request-এর জন্য database session বানানোর factory।

লাইন	ব্যাখ্যা
	expire_on_commit=False থাকায় commit-এর পরে object field পড়া যায়।
26-28	get_db dependency session খুলে route শেষ হলে context manager দিয়ে বন্ধ করে।
30-39	init_db app startup-এ table create এবং পুরোনো users/tasks table-এ নতুন column না থাকলে যোগ করে।
34	users.role default user যোগ করে।
35-37	পুরোনো tasks table-এ project_id, assigned_to_id, created_by_id যোগ করে।

এই ALTER TABLE IF NOT EXISTS পদ্ধতি শেখার/local project-এর জন্য সুবিধাজনক। বাস্তব production project-এ Alembic migration ব্যবহার করা ভালো, কারণ version history ও rollback পাওয়া যায়।

8. models.py: database table design

SQLAlchemy model-এর প্রতিটি class একটি database table। Column হলো table-এর field; ForeignKey দুটি table-র সম্পর্ক বোঝায়।

লাইন	Model / field	কেন দরকার
6-14	User	username/email unique; hashed_password নিরাপদে password রাখে; is_active দিয়ে ban/deactivate; role দিয়ে admin/user।
17-23	Project	title, description ও owner_id রাখে। owner_id হল project_owner নির্ধারণের source।
26-33	ProjectMember	কোন user কোন project-এর member এবং সে edit করতে পারবে কি না রাখে।
28	UniqueConstraint(project_id, user_id)	একজনকে একই project-এ duplicate member হিসেবে যোগ হওয়া আটকায়।
31-32	ForeignKey + CASCADE	Project বা user delete হলে তার membership automatic delete হতে পারে।
36-45	Task	project_id task কোন project-এ; assigned_to_id কার task; created_by_id কে task create করেছে।
48-54	SystemLimit	Admin-defined three global limits এক row-তে রাখে; id=1 singleton convention।

9. schemas.py: input validation ও safe response

Schema database model নয়। এটি API-তে কী input নেওয়া যাবে এবং response-এ কী দেখা যাবে তা নির্ধারণ করে। তাই hashed_password কখনো UserResponse-এ নেই।

লাইন	ব্যাখ্যা
1	BaseModel Pydantic schema base; EmailStr email validation; Field minimum/maximum validation; ConfigDict ORM object থেকে response বানাতে সাহায্য করে।

লাইন	ব্যাখ্যা
4-14	UserBase reuse করে; UserResponse id, active status, role দেয় এবং from_attributes=True দিয়ে SQLAlchemy User serialize করে।
17-18	Registration password অন্তত 8 character হতে হবে।
25-36	Login input এবং Login response। access_token ও token_type=bearer Swagger Authorization-এর জন্য দরকার।
39-50	Project create/response schemas। empty title আটকাতে min_length=1।
53-68	Member add ও can_edit update schemas।
71-93	Task create/update/response schemas। PATCH update-এ field optional; response relation ids দেখায়।
96-105	Admin limits input-এর প্রতিটি value minimum 1; response-এ id-ও আসে।

10. routers/auth.py: registration ও login

লাইন	কী ঘটে
4-11	FastAPI routing, database query/session, User/schema এবং security helpers import করা হয়।
13	APIRouter prefix=/auth; ফলে route দুটি /auth/register এবং /auth/login। tags Swagger UI-তে group বানায়।
16-21	POST register route। response model password বাদ দিয়ে safe user information ফেরত দেয়; status 201 মানে create হয়েছে।
31-37	Username duplicate query। একই username হলে 400 error।
40-46	Email duplicate query। একই email হলে 400 error।
49	Plain password hash_password-এ পাঠানো হয়; plain password database-এ যায় না।
52-59	User object বানিয়ে add, commit, refresh করা হয়। refresh নতুন id/default role আনে।
64-69	POST login route; response-এ JWT token থাকবে।
78-85	Username দিয়ে user নেয় এবং password verify করে। ভুল username/password-এ একই generic 401 দেয়, তাই account enumeration কঠিন হয়।
87-91	Deactivated account login করতে পারবে না; 403 ও temporarily banned message দেয়।
93-99	User id-কে sub claim করে create_access_token চালানো হয় এবং access_token response-এ যায়।

11. routers/admin.py: শুধু admin-এর ক্ষমতা

লাইন	ব্যাখ্যা
1-9	Router, database, role dependency, limits helper ও response schema import।
11	সব admin route /admin prefix পায়।

লাইন	ব্যাখ্যা
14-19	GET /admin/users require_role('admin') pass না করলে function body চলবেই না।
22-34	PATCH /admin/users/{id}/deactivate is_active=False করে commit/refresh করে।
37-49	PATCH activate আবার is_active=True করে।
52-65	DELETE user নিজের account নিজে delete নিষিদ্ধ; অন্য user থাকলে delete ও commit।
68-73	GET /admin/limits limits row না থাকলে get_limits default row তৈরি করে।
76-87	PUT /admin/limits schema-validated values model_dump থেকে field-by-field update করে।

নোট: কোনো user-এর linked project/task থাকলে database foreign-key policy অনুযায়ী delete বাধাগ্রস্ত হতে পারে। বাস্তব system-এ delete-এর আগে transfer/archive policy ঠিক করা উচিত।

12. core/limits.py: default limit নিশ্চিত করা

লাইন	ব্যাখ্যা
1-3	AsyncSession ও SystemLimit model import।
6	get_limits একই function সব route থেকে call করা যায়।
7	Always id=1 row খোঁজা হয়; এটিই global settings row।
8-12	Row না পেলে defaults দিয়ে insert, commit ও refresh করা হয়।
13	পরের code এই limit object ব্যবহার করে।

13. routers/projects.py: ownership ও membership

লাইন	ব্যাখ্যা
1-15	API, SQL query, current user, limits, DB, models এবং project schemas import।
17	সব project endpoint /projects prefix পায়।
20-24	get_project_or_404 helper project না থাকলে consistent 404 দেয়।
27-33	get_membership project_id + user_id দিয়ে membership নেয়। scalar_one_or_none duplicate থাকলে error দেয়; uniqueness থাকায় safe।
36-38	require_project_owner admin অথবা actual owner ছাড়া member permission change করা যায় না।
41-56	POST /projects authenticated user project create করে এবং owner_id নিজের id পায়। Admin ছাড়া অন্যদের max_projects_per_user limit count করা হয়।
59-70	GET /projects admin সব দেখে; অন্যরা নিজের owner project অথবা ProjectMember হিসেবে যুক্ত project দেখে। distinct duplicate row আটকায়।
73-83	GET one project admin/owner না হলে member হতে হবে।
86-105	POST member শুধু owner/admin পারে; owner-কে member করা, missing user, duplicate membership

লাইন	ব্যাখ্যা
	সব validate করা হয়।
108-124	PATCH member permission owner/admin can_edit true/false করে।

14. routers/tasks.py: task visibility, edit permission ও limits

লাইন	ব্যাখ্যা
15-19	can_edit_project admin ও owner সবসময় edit পারে; member কেবল can_edit=True হলে পারে।
22-28	can_view_task admin সব task; owner নিজের project-এর সব; সাধারণ user শুধু assigned_to_id নিজের হলে দেখে।
31-35	get_task_or_404 repeated task lookup/error logic এক জায়গায় রাখে।
38-43	validate_assignee assignee active user হতে হবে এবং project owner বা member হতে হবে।
46-78	POST project task edit permission check, assignee validation, user task count এবং project task count check শেষে task save করে।
58-67	দুই limit check: creator-এর max_tasks_per_user এবং project-এর max_tasks_per_project।
81-91	GET project tasks admin/owner সব task পায়; অন্যরা SQL WHERE assigned_to_id=current user পায়।
94-103	GET one task can_view_task false হলে 403।
106-126	PATCH task linked project নিয়ে editor permission নেয়; provided fields only update করে; assignee বদলালে আবার validate করে।
129-143	DELETE task admin/owner/edit-permission ছাড়া কেউ delete করতে পারে না।

15. main.py: application assembly

লাইন	ব্যাখ্যা
1	asynccontextmanager lifespan function বানানোর জন্য।
3	FastAPI main application class import।
5-9	Database initializer এবং প্রতিটি router import।
12-15	Server start হলে init_db() চলে; তাই table/migration আগে নিশ্চিত হয়। yield-এর পরে shutdown code রাখা যায়।
18	App title Swagger/OpenAPI docs-এ দেখায় এবং lifespan connect করে।
19-22	চারটি router attach করে: auth, admin, projects, tasks।
25-27	Simple health/root route; server alive আছে কি না পরীক্ষা করা যায়।

16. Role এবং permission matrix

Action	Admin	Project owner	Project member (can_edit=True)	Assigned user
সব user list / deactivate / delete	হ্যাঁ	না	না	না
System limits change	হ্যাঁ	না	না	না
Project-এর সব task দেখা	হ্যাঁ	হ্যাঁ	না	না
নিজের assigned task দেখা	হ্যাঁ	হ্যাঁ	নিজেরটি	হ্যাঁ
Member add / permission change	হ্যাঁ	হ্যাঁ	না	না
Task create / update / delete	হ্যাঁ	হ্যাঁ	হ্যাঁ	না

এখানে project_owner আলাদা global role নয়; যে user-এর Project.owner_id ওই project-এর সঙ্গে মিলে যায়, সে ওই project-এর owner। এটি per-project ownership design।

17. Swagger-এ demo করার ধাপ

ধাপ	কাজ
1	Server চালান: <code>python -m uvicorn main:app --reload</code>
2	<code>http://127.0.0.1:8000/docs</code> খুলুন।
3	POST /auth/register দিয়ে কয়েকটি user বানান; password অন্তত 8 character।
4	PostgreSQL-এ একজনকে admin করুন: <code>UPDATE users SET role='admin' WHERE username='admin_user';</code>
5	POST /auth/login থেকে access_token নিন।
6	Swagger-এর Authorize-এ Bearer <access_token> দিন।
7	Admin token দিয়ে GET/PUT /admin/limits এবং user deactivate test করুন।
8	Normal user token দিয়ে POST /projects; সে owner হবে।
9	Owner POST /projects/{id}/members দিয়ে member যোগ এবং PATCH দিয়ে can_edit true দিন।
10	Owner/editor POST /projects/{id}/tasks দিয়ে task member-কে assign করুন।
11	Assigned user token দিয়ে GET /projects/{id}/tasks চালিয়ে শুধু নিজের task দেখুন।

18. অন্যকে বোঝানোর জন্য 60-second explanation

এই project-এ প্রথমে user register করলে password Argon2 hash হয়ে database-এ যায়। Login-এর সময় password verify হলে server 60 মিনিটের JWT token দেয়। প্রতিটি protected API ওই token থেকে user id বের করে database-এ user active কি না দেখে। Admin route-এ admin role বাধ্যতামূলক। Project create করা user automatically owner হয়। Owner project member যোগ করে এবং can_edit permission দিতে পারে। Task project-এর সঙ্গে, assignee-এর সঙ্গে এবং creator-এর সঙ্গে

যুক্ত থাকে। তাই owner সব task দেখে, কিন্তু সাধারণ user SQL filter-এর কারণে শুধু নিজের assigned task দেখে। সবশেষে admin SystemLimit table-এ project/task count limit পরিবর্তন করতে পারে।

19. গুরুত্বপূর্ণ নিরাপত্তা ও পরের উন্নয়ন

- SECRET_KEY production-এ environment secret manager-এ রাখুন; .env commit করবেন না।
- echo=True শুধু learning/debug-এর জন্য; production-এ False করুন।
- Production database schema change-এর জন্য Alembic migration ব্যবহার করুন।
- User delete-এর আগে owned project/task transfer, archive অথবা cascade policy business requirement অনুযায়ী ঠিক করুন।
- Refresh token, logout/token deny-list, rate limiting, audit log এবং stronger password policy পরে যোগ করা যায়।

20. আজকের verification

Check	Result
Python syntax compile	main.py, db.py, models.py, schemas.py, core ও সব router compile হয়েছে।
JWT helper	Token create/decode test pass করেছে।
FastAPI routes	Expected auth/admin/projects/tasks routes registered হয়েছে।
PostgreSQL startup schema	Database connection ও schema initialization successful হয়েছে।

এই নথিটি 22 September 2026 তারিখে বর্তমান codebase অনুসারে তৈরি করা হয়েছে। Code পরিবর্তন হলে line number ও behavior পুনরায় যাচাই করে এই guide update করতে হবে।